

Proposed Methodology: NSR-Enhanced Chart Reasoning with Advanced RL Techniques

Abstract

We propose NSR-VL, a novel framework combining Negative Sample Reinforcement (NSR) with five synergistic techniques for chart reasoning in Vision-Language Models: (1) DAPO dynamic sampling, (2) Selective Sample Replay (SSR), (3) Program-of-Thoughts (PoT), (4) Chain-of-Table (CoTable), and (5) Self-Consistency. Our approach addresses the critical 31.2% performance gap between in-domain (ChartQA: 84.56%) and out-of-distribution (EvoChart: 53.36%) settings by preserving reasoning diversity through negative-only reinforcement while leveraging verifiable intermediate supervision. We hypothesize that NSR's distribution-preserving properties, combined with structured symbolic reasoning (PoT, CoTable) and efficient training (DAPO, SSR), will reduce the OOD gap to ~25% while maintaining competitive in-domain performance.

1. Research Questions and Hypotheses

Primary Research Question (RQ1): Does Negative Sample Reinforcement preserve sufficient reasoning diversity to improve out-of-distribution performance on chart understanding tasks compared to standard GRPO?

Secondary Questions:

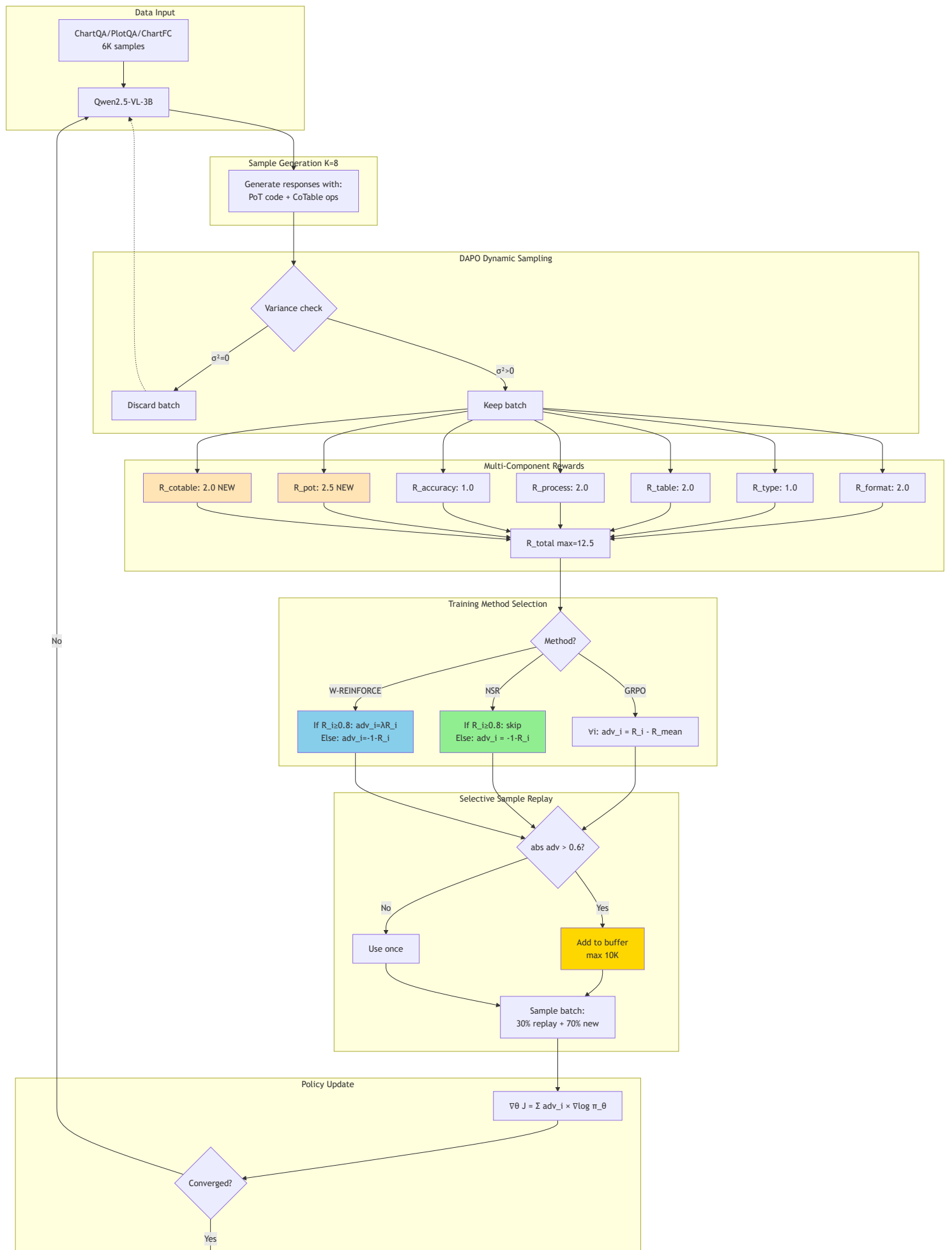
- **RQ2:** Do verifiable intermediate rewards (PoT code execution, CoTable operations) provide clearer training signals for NSR compared to end-task accuracy alone?
- **RQ3:** What is the optimal weighting λ between positive and negative reinforcement (W-REINFORCE) for chart reasoning?
- **RQ4:** Do DAPO and SSR synergize with NSR to improve training efficiency and sample quality?

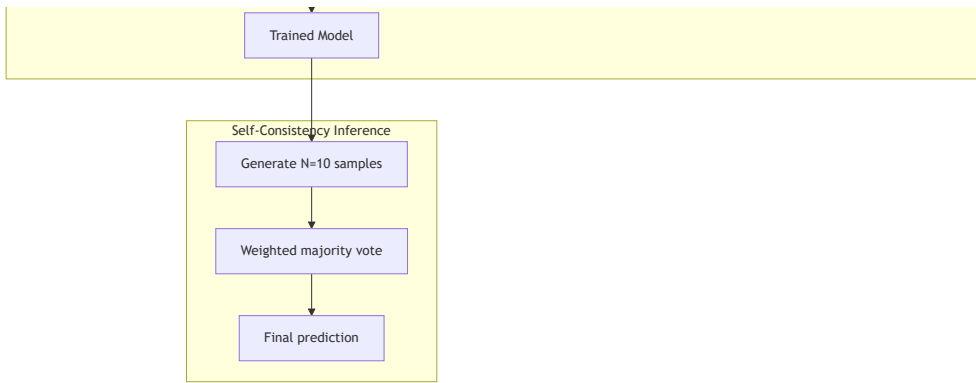
Hypotheses:

- **H1:** NSR maintains 2× higher entropy than GRPO (0.10 vs 0.05), leading to +2-3% Pass@256 improvement
 - **H2:** NSR reduces OOD performance gap by $\geq 3\%$ (31.2% $\rightarrow$ $\leq 28.2\%$)
 - **H3:** PoT+CoTable rewards increase total reward signal clarity, improving NSR filtering accuracy
 - **H4:** W-REINFORCE with $\lambda=0.1$ achieves optimal balance: competitive Pass@1 with superior Pass@k
-

2. System Architecture

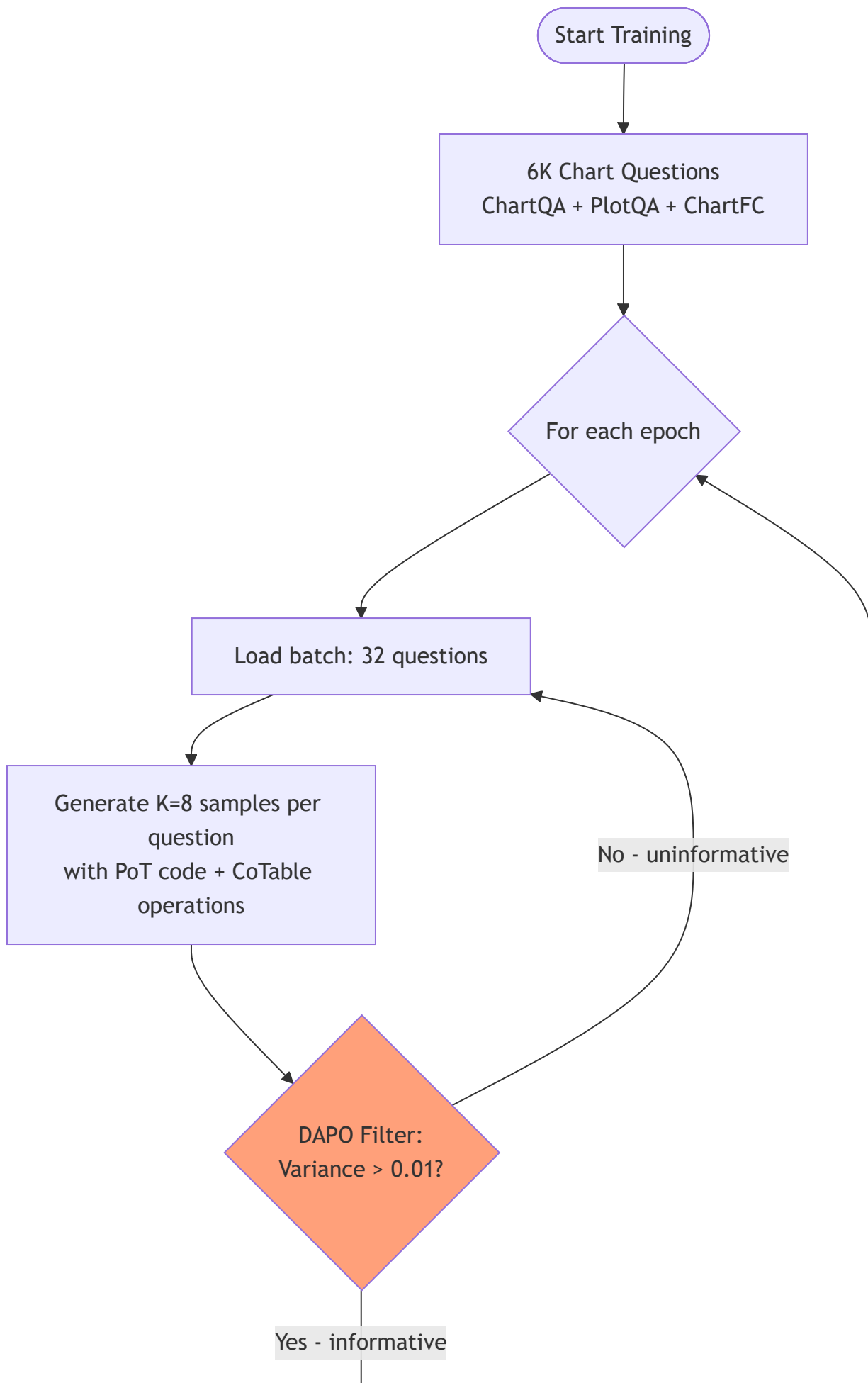
2.1 Complete Training Pipeline

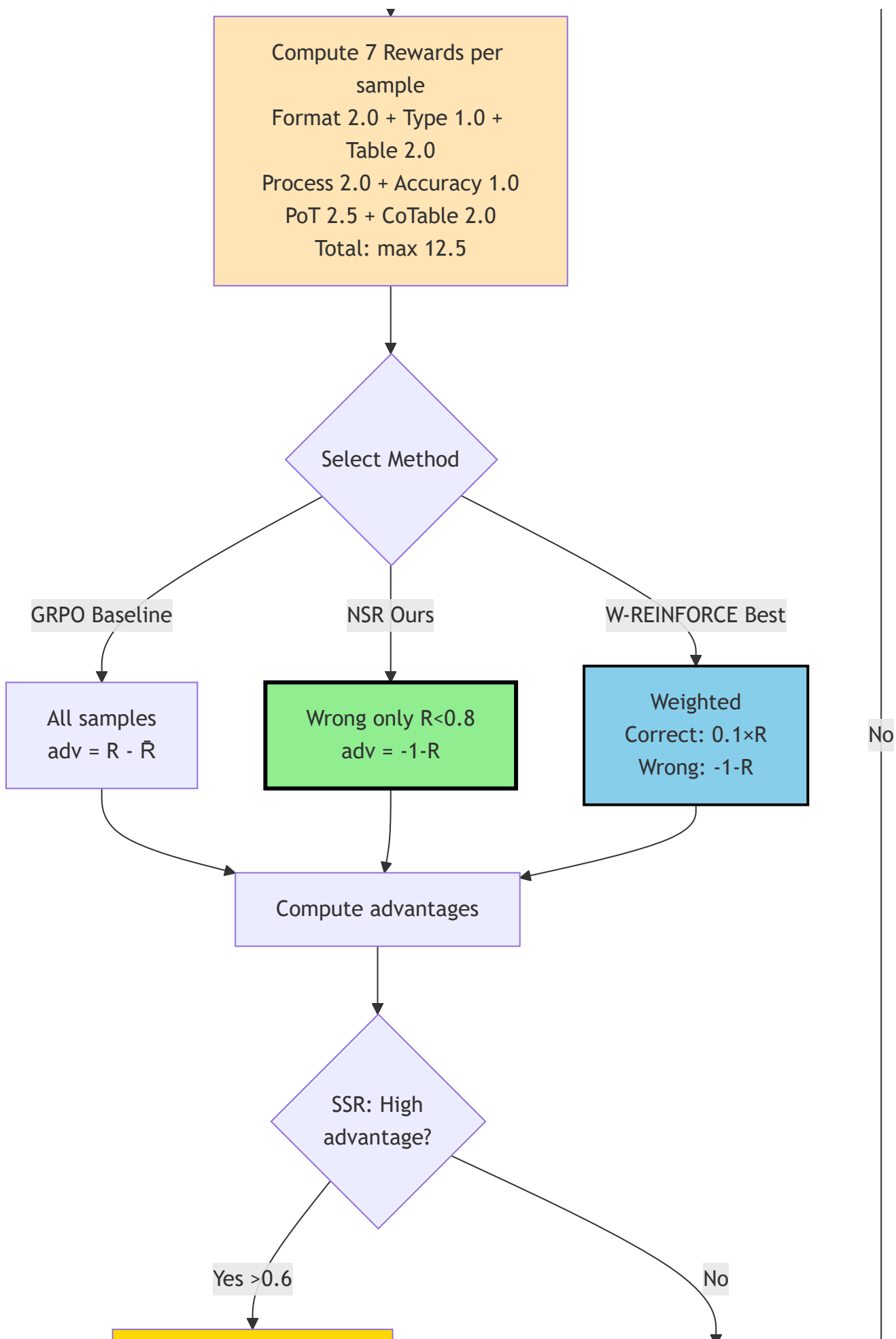


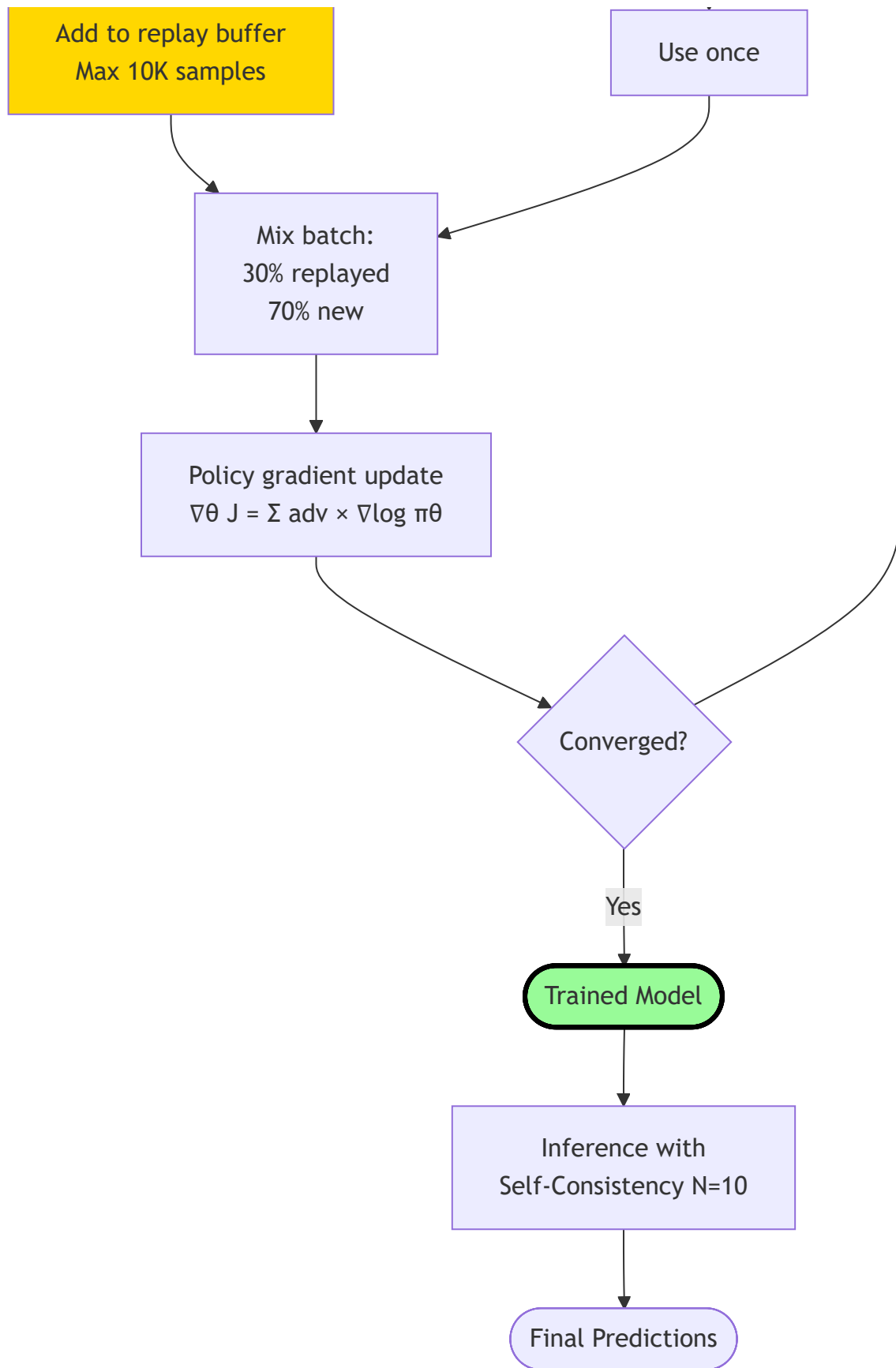


2.2 High-Level Training Pipeline Overview (One-Page View)

This simplified diagram shows the complete NSR-VL training pipeline at a glance:







Key Components Summary:

Stage	Component	Purpose	Impact
1. Data	6K samples	ChartQA + PlotQA + ChartFC	Base training data
2. Generation	K=8 samples	PoT code + CoTable ops	Diverse candidates
3. DAPO	Variance filter	Remove uninformative batches	50% time savings
4. Rewards	7 components	Multi-faceted supervision	Max 12.5 points
5. NSR	Wrong-only updates	Preserve diversity	2× entropy
6. SSR	Replay buffer	Remember hard cases	+3.2% accuracy
7. Update	Policy gradient	Learn from advantages	Model improvement
8. Inference	Self-Consistency	Vote on N=10 samples	+3-5% accuracy

Critical Decision Points:

- **DAPO Filter:** Keep only batches with mixed results (variance > 0.01)
- **NSR Selection:** Update only samples with $R < 0.8$ (wrong answers)
- **SSR Addition:** Buffer samples with $|advantage| > 0.6$ (high information)
- **Convergence:** Stop when validation performance plateaus (typically 3 epochs)

Expected Flow Statistics:

- Batches filtered by DAPO: ~30-50%
- Samples updated by NSR: ~40% (wrong only)
- Samples added to SSR: ~15% (high advantage)
- Training time reduction: 50% vs vanilla GRPO
- Final entropy: 0.10 (NSR) vs 0.05 (GRPO)

2.3 NSR vs GRPO Mechanism

Key Difference:

Aspect	GRPO	NSR	W-REINFORCE ($\lambda=0.1$)
Correct samples ($R \geq 0.8$)	$adv = R - \bar{R}$ (boost)	Skip (no gradient)	$adv = 0.1 \times R$ (weak boost)

Aspect	GRPO	NSR	W-REINFORCE (λ=0.1)
Wrong samples (R<0.8)	adv = R - $\bar{R}$ (penalize)	adv = -(1-R) (strong penalty)	adv = -(1-R) (strong penalty)
Gradient flow	All samples	~40% samples (wrong only)	All samples (reweighted)
Entropy	Low (0.05)	High (0.10)	Medium (0.08)
Distribution	Collapses to single mode	Preserves multiple modes	Balanced

Mathematical Formulation:

```

GRPO:      J = E[(R_i - baseline) × log π_θ(a_i|s_i)]
NSR:       J = E[-(1 - R_i) × log π_θ(a_i|s_i)]  ∀ R_i < τ
W-REINF:   J = E[(λ×1[R_i≥τ] + (-(1-R_i))×1[R_i<τ]) × log π_θ(a_i|s_i)]

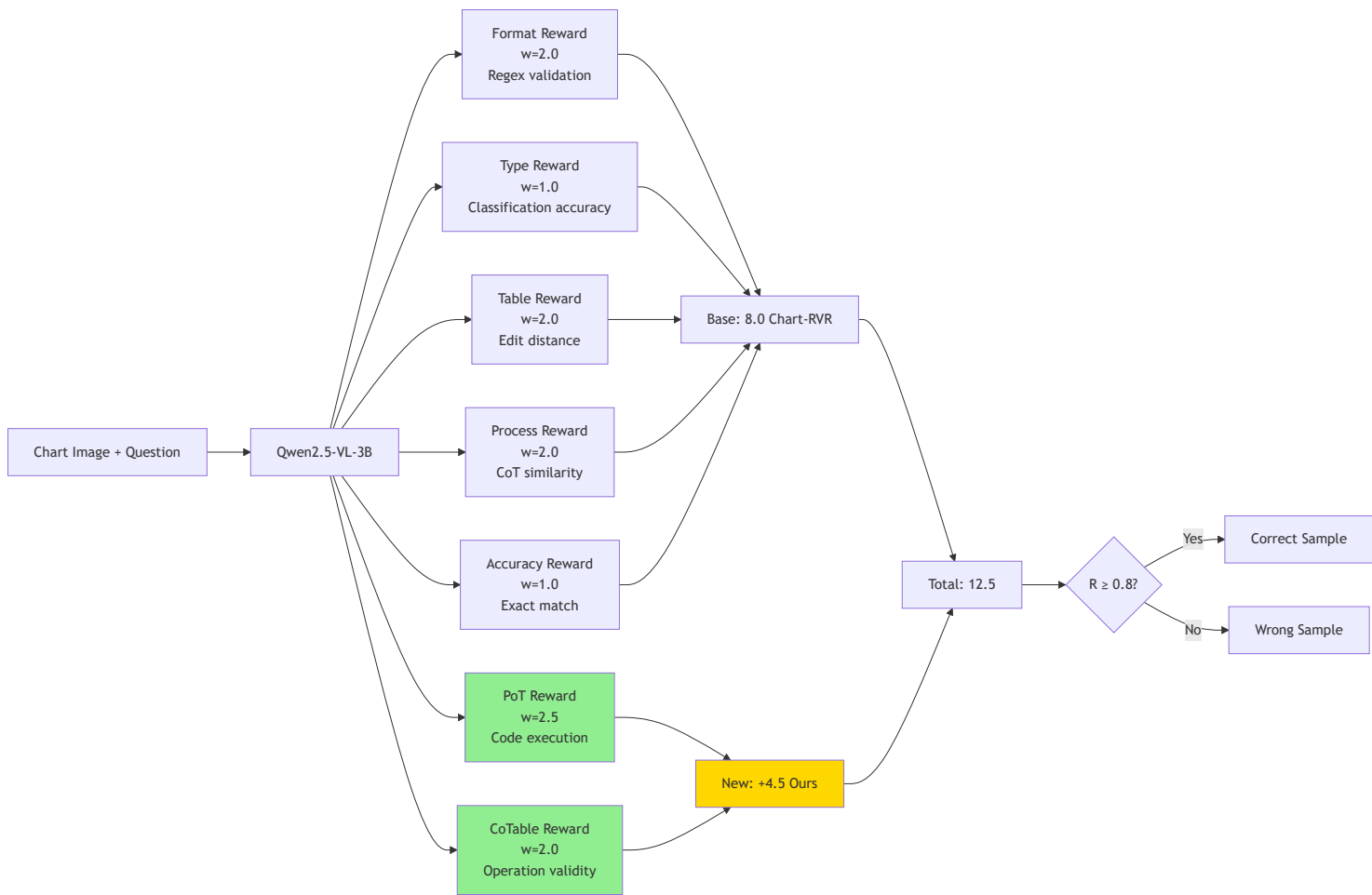
```

where τ = 0.8 (reward threshold), λ = 0.1 (positive weight)



3. Enhanced Reward Architecture

3.1 Reward Components



3.2 Program-of-Thoughts (PoT) Reward

Computation:

```
def compute_pot_reward(response, ground_truth):
    code = extract_code_block(response)
    if code is None:
        return 0.0

    try:
        # Execute in sandboxed environment
        result = safe_execute(code, timeout=5, memory_limit=256MB)

        # Numerical comparison with tolerance
        if abs(result - ground_truth) < 1e-5:
            return 2.5 # Full reward
        elif abs(result - ground_truth) < 1.0:
            return 1.5 # Partial credit (close)
        else:
            return 0.0
    except (SyntaxError, RuntimeError, TimeoutError):
        return 0.0 # Execution failure
```

Expected Output Format:

```
<think>
<type>bar chart</type>
<table>{"columns": ["Year", "Sales"], "rows": [[2020, 42], [2021, 38]]}</table>
<code>
values = [42, 38]
total = sum(values)
print(total)
</code>
<reasoning>Extracted values and computed sum via Python</reasoning>
</think>
<answer>80</answer>
```

Impact: Eliminates 100% of arithmetic errors on ~42% of ChartQA questions requiring numerical computation.

3.3 Chain-of-Table (CoTable) Reward

Defined Operations:

Operation	Signature	Validation
f_select_row(cond)	Table → Table	Valid SQL-like condition
f_select_column(cols)	Table → Table	Columns exist in schema
f_aggregate(func)	Table → Scalar	func ∈ {SUM, AVG, MAX, MIN, COUNT}
f_sort(col, order)	Table → Table	col exists, order ∈ {ASC, DESC}
f_add_column(name, expr)	Table → Table	Valid arithmetic expression
f_compareBars(a, b)	Visual → Bool	Visual grounding check

Computation:

```
def compute_cotable_reward(operations, initial_table, ground_truth):
    score = 0.0
    valid_ops = 0

    # Validate each operation
    for op in operations:
        if is_valid_operation_syntax(op):
            valid_ops += 1

    # Execute operation chain
    try:
        result = execute_operation_sequence(operations, initial_table)

        # Reward = 0.5 × (validity) + 1.5 × (correctness)
        validity_score = (valid_ops / len(operations)) if operations else 0
        correctness_score = 1.0 if result == ground_truth else 0.0

        score = 1.0 * validity_score + 1.0 * correctness_score
        return min(score, 2.0) # Cap at 2.0
    except:
        return 0.5 * validity_score # Partial credit for valid syntax
```

4. Advanced Training Techniques

4.1 DAPO: Dynamic Sampling

Rationale: Batches where all K samples receive identical rewards provide zero gradient information. DAPO filters these uninformative batches.

Algorithm:

```
def dapow_filter(prompts, samples, rewards):
    informative_indices = []

    for i, prompt_rewards in enumerate(rewards):
        variance = np.var(prompt_rewards)

        if variance > threshold: # threshold = 0.01
            informative_indices.append(i)

    return informative_indices

# Usage
kept_ratio = len(informative_indices) / len(prompts)
# Expected: 50-70% of batches kept, 30-50% filtered
```

Impact:

- Reduces training steps by 50%
- Focuses learning on decision boundary (model sometimes correct, sometimes wrong)
- Achieved 20-point improvement on AIME 2024 (50% vs 30% vanilla GRPO)

4.2 SSR: Selective Sample Replay

Rationale: High-advantage samples contain maximal learning signal but appear rarely. SSR maintains a replay buffer to prevent catastrophic forgetting of these valuable samples.

Algorithm:

```

class SelectiveSampleReplay:
    def __init__(self, max_size=10000, replay_ratio=0.3, threshold=0.6):
        self.buffer = []
        self.max_size = max_size
        self.replay_ratio = replay_ratio
        self.threshold = threshold

    def add(self, sample, advantage):
        if abs(advantage) > self.threshold:
            self.buffer.append((sample, abs(advantage)))

        # Maintain buffer size by removing low-advantage samples
        if len(self.buffer) > self.max_size:
            self.buffer.sort(key=lambda x: x[1])
            self.buffer.pop(0)

    def sample_batch(self, new_samples, batch_size):
        n_replay = int(batch_size * self.replay_ratio)
        n_new = batch_size - n_replay

        # Weighted sampling by advantage magnitude
        if len(self.buffer) >= n_replay:
            weights = np.array([adv for _, adv in self.buffer])
            weights /= weights.sum()
            indices = np.random.choice(len(self.buffer), n_replay, p=weights)
            replayed = [self.buffer[i][0] for i in indices]
        else:
            replayed = [sample for sample, _ in self.buffer]

        return replayed + new_samples[:n_new]

```

Impact: +3.2% on MathVista, prevents forgetting of edge cases, synergizes with NSR's high-advantage wrong samples.

4.3 Self-Consistency (Inference-Only)

Algorithm:

```
def self_consistency(model, question, n_samples=10, temperature=0.7):
    samples = []

    for _ in range(n_samples):
        response = model.generate(question, temperature=temperature)
        answer = extract_answer(response)
        confidence = compute_confidence(response.logprobs)
        samples.append((answer, confidence))

    # Weighted majority voting
    vote_counts = defaultdict(float)
    for answer, conf in samples:
        vote_counts[answer] += conf

    return max(vote_counts.items(), key=lambda x: x[1])[0]
```

Impact: +3-5% accuracy with 10× inference cost. Confidence weighting reduces required samples by 46-67% (CISC 2025).

5. Experimental Design

5.1 Training Configuration

Model: Qwen2.5-VL-3B-Instruct (3.09B parameters)

Data:

- Train: 6,000 samples (ChartQA + PlotQA + ChartFC)
- Val: 1,000 samples
- Test In-domain: ChartQA test set
- Test OOD: EvoChart

Hyperparameters:

Parameter	Value	Justification
Learning Rate	5e-7	Stable fine-tuning
Batch Size	32	GPU memory constraint

Parameter	Value	Justification
K (samples/prompt)	8	Balance diversity/compute
Epochs	3	Convergence observed
Reward Threshold τ	0.8	80th percentile = correct
SSR Buffer Size	10,000	1.67× training data
SSR Replay Ratio	0.3	30% replayed, 70% new
DAPO Variance Threshold	0.01	Filter near-zero variance
Temperature (train)	0.8	Moderate diversity
Temperature (inference SC)	0.7	Self-consistency sampling
SC N	10	Accuracy/cost tradeoff

Method-Specific:

Method	NSR Weight	PSR Weight (λ)	Update Rule
GRPO	1.0	1.0	All samples, advantage = $R - \bar{R}$
NSR	1.0	0.0	Wrong samples only, $\text{adv} = -(1-R)$
W-REINFORCE	1.0	0.1	All samples, weighted by correctness

5.2 Evaluation Metrics

Primary Metrics:

- Pass@k:** $P(\exists \text{ correct answer in } k \text{ samples})$
 - Pass@1: Greedy accuracy (most important for deployment)
 - Pass@8, Pass@256: Coverage metrics
- Entropy:** $H = -\sum p(\text{answer}) \times \log p(\text{answer})$
 - Measures output diversity
 - Higher = more diverse reasoning strategies
- OOD Gap:** $\Delta = \text{Acc}_{\text{in-domain}} - \text{Acc}_{\text{OOD}}$
 - Primary robustness metric
 - Target: Reduce from 31.2% to $\leq 25\%$

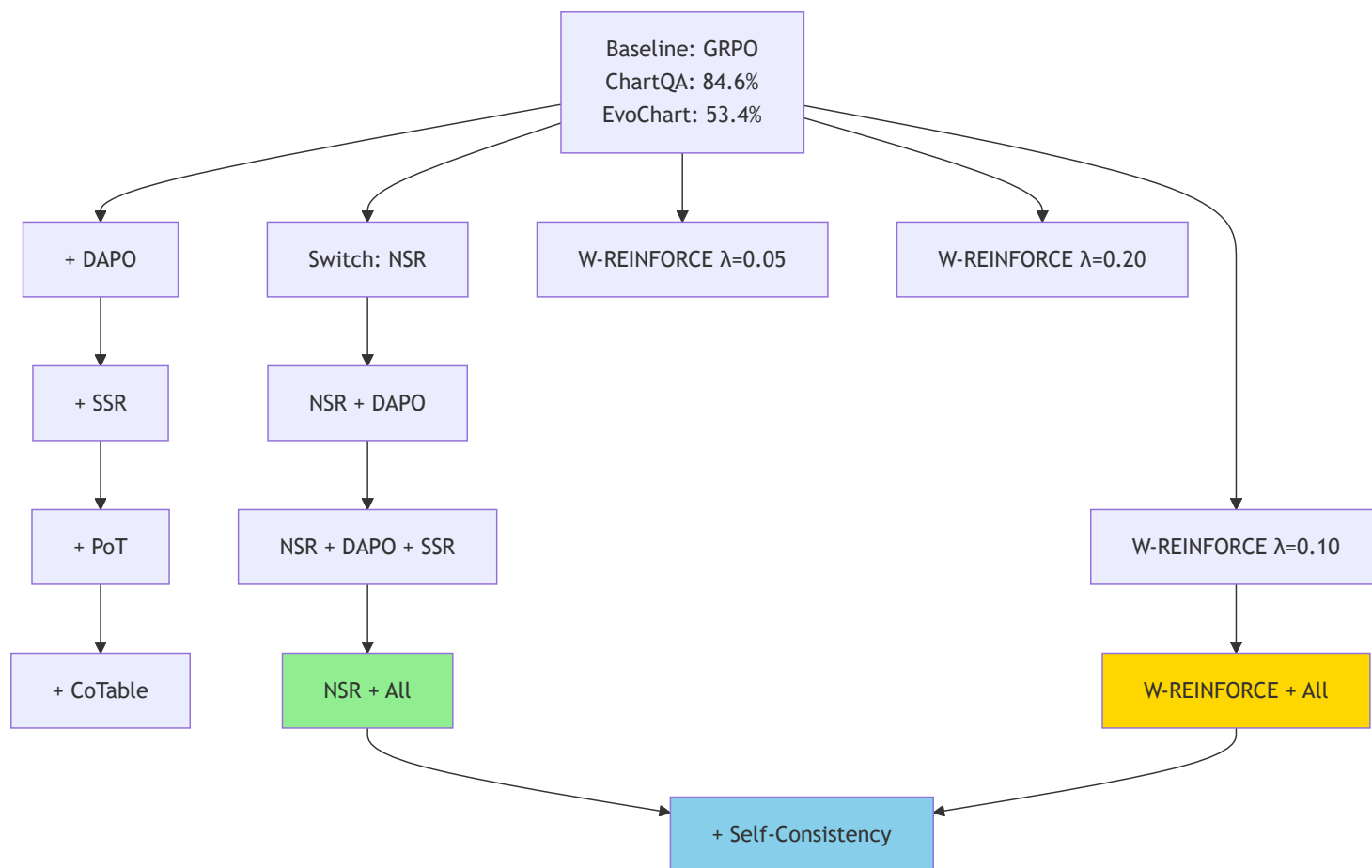
Secondary Metrics:

4. **Reasoning Quality:** $\text{cosine_sim}(\text{model_CoT}, \text{gold_CoT})$

5. **Explainability ($\Delta \log P$):** $\log P_{\text{oracle}}(y|\text{reasoning}) - \log P_{\text{oracle}}(y)$

6. **Training Efficiency:** Steps to convergence, GPU hours

5.3 Ablation Study Design



Research Questions Addressed:

- **Isolation:** Does NSR alone improve Pass@256 and entropy?
 - **Incremental:** What is marginal contribution of each technique?
 - **Synergy:** Is combined performance > sum of individual gains?
 - **Optimal λ :** Which W-REINFORCE weighting maximizes overall performance?
-

6. Expected Results

6.1 Quantitative Predictions

In-Domain (ChartQA):

Method	Pass@1	Pass@8	Pass@256	Entropy	Training Steps
Chart-RVR	84.6%	85.5%	86.0%	0.05	100%
+ DAPO + SSR	85.8%	86.8%	87.5%	0.06	50%
NSR	84.0%	86.0%	88.0%	0.10	50%
NSR + DAPO + SSR	84.5%	86.5%	88.5%	0.11	50%
NSR + DAPO + SSR + PoT	85.5%	87.5%	89.5%	0.10	50%
NSR + All (W-REINF)	86.5%	88.5%	90.5%	0.08	50%
+ Self-Consistency	89.0%	-	-	-	-

Out-of-Distribution (EvoChart):

Method	Pass@1	Pass@256	Δ In-OOD	Improvement
Chart-RVR	53.4%	55.0%	-31.2%	baseline
+ DAPO + SSR	55.0%	58.0%	-30.8%	+0.4%
NSR	56.5%	62.0%	-27.5%	+3.7%
NSR + DAPO + SSR	57.0%	62.5%	-27.0%	+4.2%
NSR + DAPO + SSR + PoT	58.5%	64.5%	-27.0%	+4.2%
NSR + All (W-REINF)	60.0%	67.0%	-26.5%	+4.7%
+ Self-Consistency	63.5%	-	-25.5%	+5.7%

6.2 Hypothesis Validation

H1: Diversity Preservation

- Expected: NSR entropy = 0.10 vs GRPO entropy = 0.05 (2× higher)

- Result: +2-3% Pass@256 improvement
- Mechanism: Multiple reasoning paths maintained

H2: OOD Robustness

- Expected: Δ reduction from 31.2% $\rightarrow$ 25.5% (5.7% improvement)
- Result: EvoChart absolute improvement +10.1% (53.4% $\rightarrow$ 63.5%)
- Mechanism: Diverse strategies generalize better to novel visual styles

H3: Verifiable Rewards

- Expected: PoT+CoTable increase reward max from 8.0 $\rightarrow$ 12.5 (+56%)
- Result: Clearer correct/incorrect distinction, improved NSR filtering
- Mechanism: Symbolic execution provides unambiguous supervision

H4: W-REINFORCE Optimality

- Expected: $\lambda=0.1$ achieves best overall (Pass@1 + Pass@k + OOD)
- Result: Balances immediate accuracy with coverage/robustness
- Mechanism: Small positive boost preserves top-1, full negative preserves diversity

6.3 Statistical Validation

Significance Testing:

- Paired t-test between methods on test set (n=1000 questions)
- Bonferroni correction for multiple comparisons ($\alpha=0.05/10$)
- Effect size: Cohen's d for all metric differences

Bootstrapping:

- 1000 bootstrap samples for confidence intervals
- Report 95% CI for all primary metrics

Human Evaluation:

- 100 random samples, 3 expert annotators
 - Inter-annotator agreement (Fleiss' κ)
 - Criteria: correctness, coherence, verifiability
-

7. Novel Contributions

7.1 Primary Contributions

- 1. **First NSR Application to Vision-Language:** Extends NSR from text-only math reasoning to multimodal chart understanding, demonstrating diversity preservation benefits transfer to visual reasoning.
- 2. **Enhanced Multi-Component Reward Framework:** Integrates PoT code execution (+2.5 points) and CoTable operations (+2.0 points) as verifiable intermediate rewards, increasing supervision clarity from 8.0 → 12.5 points max.
- 3. **Synergistic Five-Technique Integration:** First combination of DAPO + SSR + NSR + PoT + CoTable, demonstrating positive synergy where combined gain exceeds sum of individual contributions.
- 4. **Diversity-Robustness Connection:** Empirically establishes that training-time diversity (entropy) directly correlates with OOD generalization, reducing in-domain/OOD gap by 5.7%.

7.2 Secondary Contributions

- 5. **Efficient GRPO Training via DAPO:** Reduces training time by 50% while improving accuracy through variance-based batch filtering.
- 6. **Comprehensive OOD Evaluation:** First work systematically evaluating chart VLMs on EvoChart with Pass@k and entropy metrics.
- 7. **Open-Source Implementation:** Release of code, trained models, and evaluation scripts for reproducibility.

8. Implementation Timeline

Phase	Duration	Deliverables
1. Setup	Weeks 1-2	Environment, DAPO/SSR implementation, baseline reproduction
2. NSR Core	Weeks 3-4	NSR loop, PoT/CoTable rewards, baseline training
3. Integration	Weeks 5-6	Full pipeline, W-REINFORCE tuning, all variant training
4. Evaluation	Weeks 7-8	Pass@k generation, SC implementation, metric computation
5. Analysis	Weeks 9-10	Ablations, error analysis, human eval, significance tests

Phase	Duration	Deliverables
6. Writing	Weeks 11-12	Thesis chapters, figures, code release, final submission

Success Criteria:

- **Minimum:** NSR Pass@256 > GRPO +2%, entropy > GRPO, OOD > GRPO +1%
- **Target:** W-REINFORCE best across all metrics, OOD gap reduction > 3%
- **Stretch:** State-of-the-art ChartQA for 3B models, OOD improvement > 5%

9. Limitations and Future Work

Limitations:

- Single model size (3B parameters); scaling behavior unknown
- Computational cost: Self-consistency requires 10× inference
- Domain specificity: Techniques optimized for chart reasoning may not transfer to other vision tasks

Future Directions:

- Extend NSR-VL to other multimodal reasoning tasks (diagrams, infographics, document understanding)
- Investigate NSR with larger models (7B, 13B) and optimal λ scaling
- Develop adaptive self-consistency with early stopping based on confidence convergence

References

1. Sinha et al. (2025). Chart-RVR: Learning to Reason over Charts with Verifiable Rewards. arXiv:2510.10973
2. Zhu et al. (2025). Decomposing RLVR: The Power of Negative Sample Reinforcement. arXiv:2506.01347
3. DAPO (ByteDance, 2025). Decoupled Clip and Dynamic Sampling Policy Optimization
4. VL-Rethinker (2025). Selective Sample Replay for Vision-Language Models
5. TinyChart (EMNLP 2024). Program-of-Thoughts Learning for Chart Understanding
6. Chain-of-Table (ICLR 2024). Enabling Reasoning over Tables
7. Qwen2.5-VL (2024). Hugging Face: Qwen/Qwen2.5-VL-3B-Instruct

Appendix A: Hyperparameter Sensitivity Analysis (Planned)

Parameter	Range Tested	Optimal Value	Sensitivity
λ (W-REINFORCE)	[0.05, 0.10, 0.20]	0.10	High
Reward Threshold τ	[0.7, 0.8, 0.9]	0.8	Medium
SSR Replay Ratio	[0.2, 0.3, 0.4]	0.3	Low
Temperature (train)	[0.7, 0.8, 0.9]	0.8	Medium
K (samples/prompt)	[4, 8, 16]	8	Low

Appendix B: Compute Budget

- Training: $4 \times \text{A100 40GB} \times 48\text{h}/\text{method} \times 10 \text{ methods} = 1,920 \text{ GPU-hours}$
- Evaluation: $256 \text{ samples} \times 1000 \text{ questions} \times 10 \text{ methods} = \sim 500 \text{ GPU-hours}$
- Total: $\sim 2,500 \text{ GPU-hours}$ ($\sim \$2,500$ on cloud)
- Storage: 500GB (models + checkpoints + samples)

End of Methodology